

Q3 • 2022

ThinkstScapes Quarterly

Brought to you by



**Most companies find out way too late
that they've been breached.**

Thinkst Canary changes this.

Canaries deploy in under 4 minutes
and require 0 ongoing admin overhead.

They remain silent until they need to chirp,
and then, you receive that single alert.

When.it.matters.

Find out why some of the smartest security teams
in the world swear by Thinkst Canary.

<https://canary.love>

Contents

Introduction	4
A year in review	5
Themes covered in this issue	6
AI/ML boosted side-channels	7
Analyzing the Feasibility and Generalizability of Fingerprinting Internet of Things Devices	8
Watching the Watchers: Practical Video Identification Attack in LTE Networks	9
Can one hear the shape of a neural network?: Snooping the GPU via Magnetic Side Channel	10
LTrack: Stealthy Tracking of Mobile Phones in LTE	11
Clever cryptography	12
IRMA's Idemix core: Understanding the crypto behind selective, unlinkable attribute disclosure	13
CryptPad: a zero knowledge collaboration platform	14
drand: publicly verifiable randomness explained	15
A dead man's full-yet-responsible-disclosure system	15
Oops... Code Execution and Content Spoofing: The First Comprehensive Analysis of OpenDocument Signatures	16
My data in your signed code	16
Can You Trust a File's Digital Signature? New Zloader Campaign exploits Microsoft's Signature Verification	16
Software analysis at scale	17
TLS-Anvil: Adapting Combinatorial Testing for TLS Libraries	18
Arbiter: Bridging the Static and Dynamic Divide in Vulnerability Discovery on Binary Programs	19
In Need of 'Pair' Review: Vulnerable Code Contributions by GitHub Copilot	20
Catch Me If You Can: Deterministic Discovery of Race Conditions with Fuzzing	21
Someone's Been Messing With My Subnormals!	22
Nifty Sundries	23
Attacking AAD abusing Synch API: The story behind \$40K in bounties	24
Towards a Tectonic Traffic Shift? Investigating Apple's New Relay Network	25
Hiding malware in Docker Desktop's secret virtual machine	26
Let's Dance in the Cache - Destabilizing Hash Table on Microsoft IIS	27
Using Trätṛ to tame Adversarial Synchronization	27
Conclusion	28

Introduction

Welcome to the Q3, 2022 edition of ThinkstScapes! This issue focuses on content released, published, or presented since the publication of the Q2 2022 quarterly release.

This edition marks a full year since ThinkstScapes was reintroduced – over the last year, editors have reviewed over 4,400 talks, papers and presentations (in addition to 1,000s of blog posts!). To mark this milestone, in the next section we reflect on our identified trends and themes, what we caught early and flagged for readers, and what we missed.

This quarter contained a large number of conferences, including the “Hacker Summer Camp”, as well as much expanded blog coverage – there should be something in here for everyone. As a reminder: we would appreciate your help in catching any interesting work that may have fallen through the cracks – any papers, presentations, or blog posts are welcome. Please send them to ts@thinkst.com!

This issue includes talks drawn from the following conferences (and almost 2,000 security blog posts):

Conference/venue name	Number of publications reviewed
USENIX Security 2022	256
Black Hat USA	101
DEF CON 30	85
A New HOPE	81
BSides Las Vegas	90
Kawaii Con	26
Pass the SALT 2022	27
The 22nd Privacy Enhancing Technologies Symposium	128
Toor Camp	65
x33fcon	18
May Contain Hackers 2022	209
HITB Singapore	19
Crikey Con 8	24
44CON	15
Circle City Con 9.0	25
RootCon 16	25
Texas Cyber Summit	111
Romhack	8
VirusBulletin Prague	48
Corn Con	40
Total	1401



As always, Thinkst Labs is happy to notify you when we release a new issue. [Sign up on the ThinkstScapes homepage](#) where you can also find a link to the audio summary of this, and all previous issues.

A year in review

This edition of ThinkstScapes marks a year since the reboot, so it is important to reflect on the trends called out in the past year (even if only to grade our editors on their predictions). Each trend highlighted in a TS issue is note-worthy: either because there is a new research area being explored, or because there is an uptick in momentum for a more well-trodden field. We explore the trends called out in the third and fourth quarters of 2021 and see what work has been done more recently; has that trend continued or has it dropped off?



TRENDS IN TS Q3'21

Into the embedded realm

– A trend of embedded or low-level research into more impactful systems as skills honed on “junk hacking” were applied to critical systems.

Defence – Real-world defences with high-quality empirical experimentation that supports the research.

Exploiting “Differences of Opinion” – Research into gaps between data recognition and parsing in complex environments where one component understands input differently than another.



TRENDS IN TS Q4'21

Making servers (over)work for fun and profit

– A resurgence of research into workload attacks focused on algorithms or data structures as well as putting more computational onus on servers that pay security dividends for clients (e.g., encrypted search or confidential computing).

The AD and Azure beast

– A trend of exploration and exploitation into Azure AD and hybrid environments.

Analyse and fix – Research into better analysis techniques that provide practical and real security improvements with fewer manual steps.

Bridging gaps and making gaps – Research exploring the OT space as well as other novel network partitions and how to bridge them.

In reviewing content for this quarter's edition, there were research publications that hit on every one of the above trends. While some of them are quite broad, the more narrowly-scoped all have clear examples of current work being done, a year later. This issue also contains some work that would have fit into the existing themes: algorithmic attacks, improved analysis tooling to explore novel vulnerability classes, and deep dives into the underlying hardware and protocols that power ubiquitous computing. In addition to the work highlighted in this edition, there were multiple talks at “Hacker Summer Camp” covering the AD/Azure beast and OT/ICS networks.

Areas that, with the benefit of hindsight, should have been called out sooner include:

- ✓ **Attacks against CI/CD pipelines**
– Not only has this been used in practice by attackers, there has also been some great work in this area dating back to the periods covered by last year's ThinkstScapes.
- ✓ **AI/ML's role in boosting low signal-to-noise ratio attacks**
– While this was called out in Q1's issue, the work performed is broader than we originally anticipated, including cross-modality side-channels as well as improving the practicality of micro-architectural/speculative attacks.
- ✓ **Increasing fuzzing's efficacy** – While improvements to fuzzers has been highlighted, the collective improvements in moving from simply finding crash-inducing memory corruption issues to multi-bug class exploration engines was underappreciated.

Research prediction is a relatively small part of what we do with TS, but we will challenge ourselves to do better. If you have thoughts or comments, we'd love to hear from you at ts@thinkst.com!

Themes covered in this issue

AI/ML BOOSTED SIDE-CHANNELS

Glimpses of this have been seen in the past, highlighting this as a field of research reaching an inflection point; this quarter had enough of a showing to be a theme in itself. Using AI/ML models to pull out faint signals from noisy side-channels can have real-world implications. This theme includes work in passively fingerprinting IoT devices, geo-locating LTE devices, reversing an ML model from GPU's magnetic fields, and even determining what videos are being streamed to LTE devices connected to the same tower.

CLEVER CRYPTOGRAPHY

Cryptography has been used extensively to protect and confirm the authenticity of data. Work highlighted in this theme shows other novel use-cases, or novel schemes that expand on how cryptography can empower users, or put them at risk. From new privacy-preserving authentication schemes and time-lock encryption, to end-to-end encrypted collaboration suites and ways that digital signatures fail, this theme covers a lot of territory.

SOFTWARE ANALYSIS AT SCALE

Software analysis is a mature field, but the work highlighted in this theme showcases larger scale analyses. By building tools that are easier to run on targets, more coverage and bugs can be discovered earlier, hopefully improving security for all. This theme includes a hybrid static and dynamic analysis suite that can run on the entire Ubuntu repository, new tools that make it easier to test TLS libraries and find race conditions, and an automated analysis of code generated by AI assistants.

NIFTY SUNDRIES

There were some papers that didn't fit into any specific theme, but were worth including. This quarter includes work on embedding malware into Docker, crashing hash tables, breaking AD and Azure synchronisation, and a deep-dive into Apple's new Private Relay.

Photo by Ian Badenhorst on Unsplash.



AI/ML boosted side-channels

- ✓ *Analyzing the Feasibility and Generalizability of Fingerprinting Internet of Things Devices*
- ✓ *Watching the Watchers: Practical Video Identification Attack in LTE Networks*
- ✓ *Can one hear the shape of a neural network?: Snooping the GPU via Magnetic Side Channel*
- ✓ *LTrack: Stealthy Tracking of Mobile Phones in LTE*

Photo by Juanita Swart on Unsplash.

Analyzing the Feasibility and Generalizability of Fingerprinting Internet of Things Devices

Authors: Dilawer Ahmed, Anupam Das, and Fareed Zaffar



[Code](#)

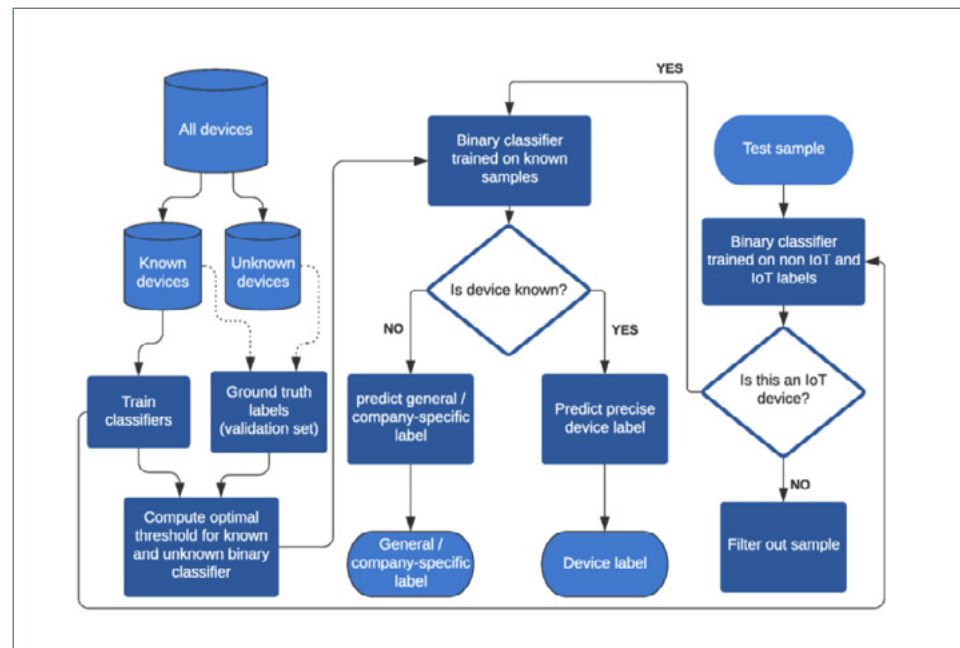


[Paper](#)

This work explored training a ML model on certain aspects of IoT network traffic and meta-data that would not be protected by encryption (e.g., timings, packet sizes, etc.) to then fingerprint those devices in network traces. The researchers were able to confidently (90%+) identify those devices from within large traffic captures with the content of all IP traffic removed (to simulate encrypted links).

With different experimental designs, the researchers were able to adjust the granularity of their detections to differentiate between different devices of the same model. When examining a device that is not explicitly trained on, the model could attempt to determine the class of device (or the role it plays). These were tracked across datasets, including the dataset generated and released on the linked GitHub page.

Figure 1: A flow-chart of the training and testing process for passively identifying an IoT device from its network traffic.



TAKEAWAYS:

- ✓ **Using a model on meta-data of encrypted network traffic** may seem difficult to passively access in practice, but many IoT devices of interest are increasingly connected over wireless networks that expose meta-data to other listening devices in the same region. Past work by James Pavur (featured in the Q3'21 ThinkstScapes) on sniffing satellite connections also provides the visibility needed to view this traffic.
- ✓ **There is ample concern regarding the privacy concerns of IoT devices from their vendors**, and this work highlights the added third-party privacy concerns. When applied to sensitive devices (e.g., internet-connected medical devices), those privacy concerns can become very real.

Watching the Watchers: Practical Video Identification Attack in LTE Networks

Authors: Sangwook Bae, Mincheol Son, Dongkwan Kim, CheolJun Park, Jiho Lee, Soeul Son, and Yongdae Kim



[Paper](#)



[Video](#)



[Website](#)

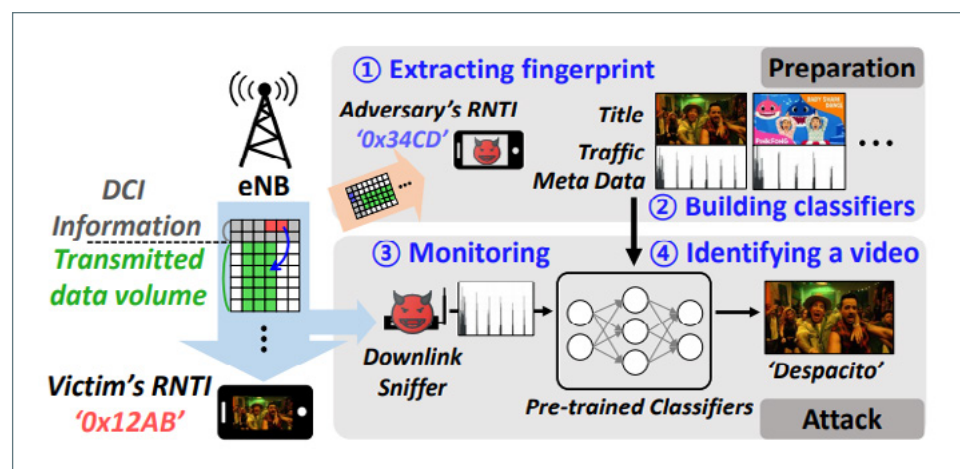
This research explored the possibility of passively monitoring traffic patterns for nearby LTE devices (same LTE cell) to determine, with high accuracy, the video being streamed by a target device. By building a classifier that can identify the video based on the traffic pattern fingerprint, those patterns can be detected and the process reversed for the target's encrypted wireless downlink. HTTP adaptive streaming, the most popular protocol for streaming video, splits up the video into fixed-duration chunks that have different sizes due to the variable bitrate encoding. Therefore each video has a unique, repeatable sequence of chunk sizes irrespective of encryption. After training a Convolutional Neural Network (CNN) on multiple video chunk sequences and a model on the metadata of the stream (chunks sent at a time), the CNN can determine both the video and the streaming platform with an accuracy of at least 90% from an LTE downlink trace for a video that it was trained on.

Due to the unencrypted metadata in LTE transmissions, traffic for a specific device can be retrieved from the downlink and collated. The CNN is used to handle noise in the traffic sequence information stemming from other network traffic on the LTE device that is combined with the video content. The research also explored combining multi-channel downlinks with multiple passive monitors as well as actively injecting data to cause a specific device to alarm audibly (using the built-in presidential alerting mechanism).

TAKEAWAY:

- ✓ **With wireless access becoming the prevailing end-user access medium**, attacks like these will become increasingly possible. Encryption of public data with recognizable traffic patterns cannot guarantee privacy if an attacker can observe those patterns. With enough training data and decision trees that combine multiple models, high-confidence privacy violations will be more likely.

Figure 2: A diagram of the high-level steps to identify the video an LTE device in the same cell is streaming.



Can one hear the shape of a neural network?: Snooping the GPU via Magnetic Side Channel

Authors: Henrique Teles Maia, Chang Xiao, Dingzeyu Li, Eitan Grinspun, and Changxi Zheng



[Paper](#)



[Slides](#)

This work explored whether there was sufficient recoverable data in the magnetic field surrounding the power cable on a GPU to infer information about the ML model it was accelerating. With a low-cost sensor, the researchers were able to train a model on the magnetic field on their own test systems and GPUs, then with high reliability infer the network structure of a model on a victim's system. This was successful against networks of varying sizes, including those used in modern applications. The inferred network parameters were then used to create a surrogate model that, once trained, was used to build a black-box adversarial AI attack. On simpler networks, the attacks succeeded in more than 85% of the trials, and even on larger and more complex networks, the surrogate-trained attack was successful more than 55% of the time. Future work needs to determine the impact of the training set on the efficacy of the attack, but a short time with passive physical access to a machine running a model can reveal a considerable amount about its design and operation.

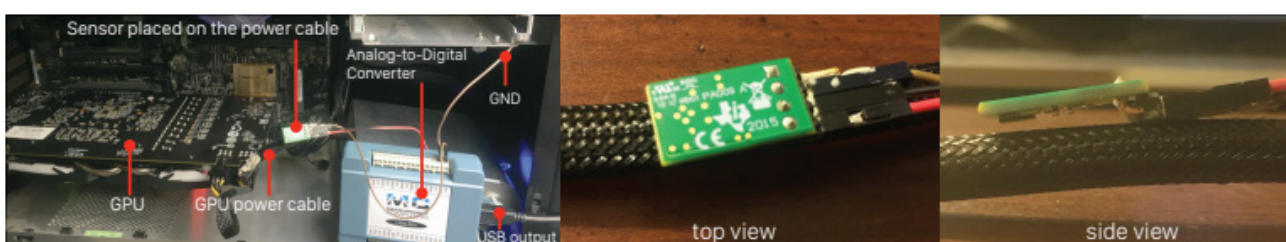
TAKEAWAYS:

- ✓ **Due to the expense in training a high-quality model**, once the model is sufficient it is likely to have a long deployment lifespan. If a short term physical side-channel can determine the network parameters, a more effective black-box surrogate can be fashioned to target traditional adversarial attacks against the model. An attacker's ability, through brief physical access, to target adversarial attacks against a model for the duration of its deployment, is a real concern that has not been sufficiently considered.
- ✓ **This work continues to highlight** how ML can be used to boost the signal from side-channels into something with practical consequences. A revisitation is due of past/historic side-channels that were looked over for being too noisy with these more powerful ML signal recovery techniques.



A short time with passive physical access to a machine running a model can reveal a considerable amount about its design and operation.

Figure 3: Views of the physical intrusion needed to gain access to the GPU's EM emissions.



LTrack: Stealthy Tracking of Mobile Phones in LTE

Authors: Martin Kotuliak, Simon Erni, Patrick Leu, Marc Röschlin, and Srdjan Čapkun



[Paper](#)



[Slides](#)

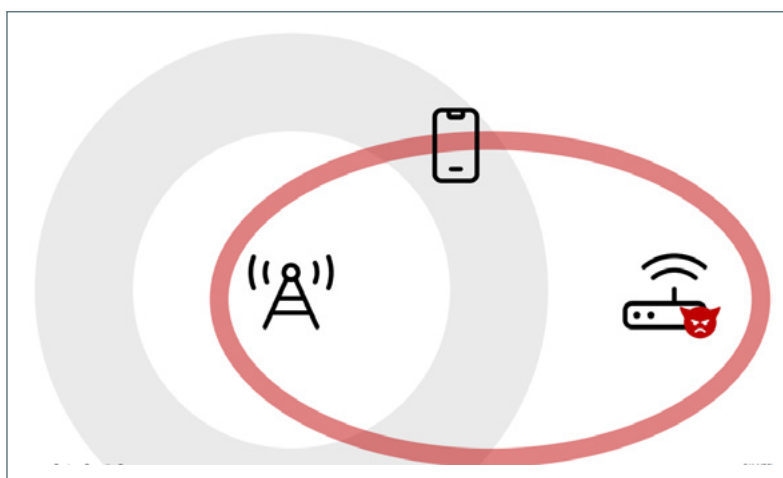
This work looks at both improving the ability to passively locate an LTE device from within the same wireless zone/cell, and explores an active attack to de-anonymise a device with a low probability of detection. Using a combination of unencrypted metadata traffic as well as LTE's strict time division scheme to support multiple devices, a passive attacker (i.e., no transmission) can locate a device to at best within 6 m. In order to prevent LTE devices from transmitting over one another, each is assigned a timeslot to communicate with the tower. However, due to the time needed for a radio signal to traverse the distance from device to tower, each device communicates with the tower to determine its distance from the tower and uses that distance to calculate a timing offset for how early to send a signal for it to arrive at the correct time.

These researchers were able to capture this timing offset from the victim device to determine a rough ring from the tower (grey circle in the below figure). The attacker then uses its own distance offset from the victim device to further refine the location accuracy with an ellipse (red in below figure). With this passive attack, attackers can locate a nearby device, but only have access to the temporary/anonymous identifier of the device, not its permanent identification number (IMSI). Past work to correlate temporary IDs to IMSIs required the deployment of a fake basestation (IMSI Catcher) that could be detected. This work uses an overshadowing attack to only broadcast over a specific message from the tower with marginally higher power, prompting the victim device to send its IMSI.

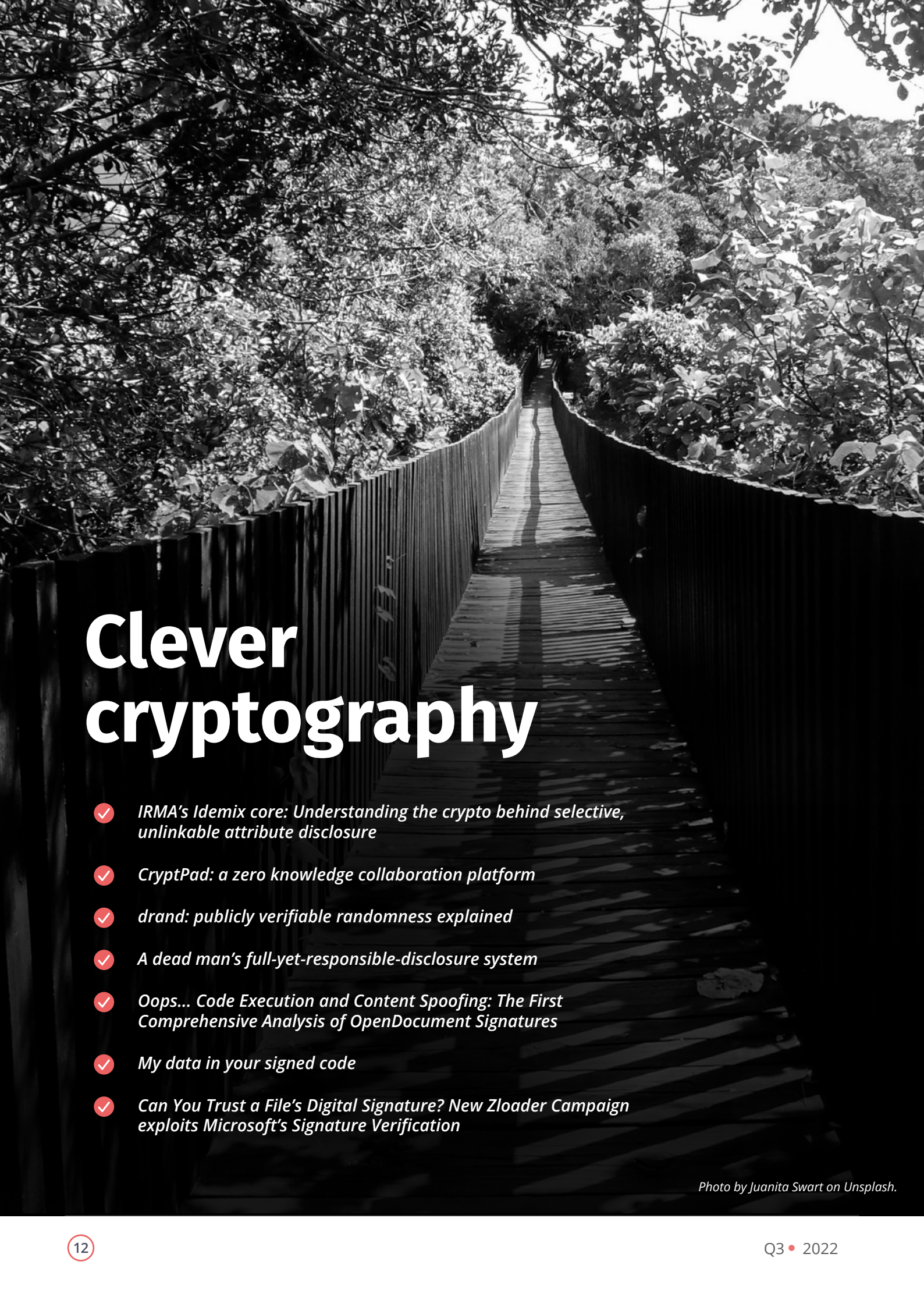
TAKEAWAYS:

✓ **Even with a highly-secured mobile OS**, privacy-leaking attacks are possible within a certain distance due to the physics of wireless communications before a single OS function is called. If your adversary is willing to put devices in the field, carrying a phone in your pocket may not be the best OPSEC.

Figure 4: A simplified figure of using timing information to passively locate an LTE handset.



✓ **Precise timing is essential to modern technology**, and it is incredible how sensitive mobile devices are when laptops and desktops have clocks that drift enormously. Time reference devices are still expensive (\$5-10k), but GNSS receivers are not – in the search for speed, security may be at risk to the attacker with the most accurate clocks.



Clever cryptography

- ✓ *IRMA's Idemix core: Understanding the crypto behind selective, unlinkable attribute disclosure*
- ✓ *CryptPad: a zero knowledge collaboration platform*
- ✓ *drand: publicly verifiable randomness explained*
- ✓ *A dead man's full-yet-responsible-disclosure system*
- ✓ *Oops... Code Execution and Content Spoofing: The First Comprehensive Analysis of OpenDocument Signatures*
- ✓ *My data in your signed code*
- ✓ *Can You Trust a File's Digital Signature? New Zloader Campaign exploits Microsoft's Signature Verification*

Photo by Juanita Swart on Unsplash.

IRMA's Idemix core: Understanding the crypto behind selective, unlinkable attribute disclosure

Authors: Maja Reissner
and Sietse Ringers



[Code](#)



[Video](#)



[Website](#)

This research presents IRMA, a digital verification ecosystem built on the separation of issuing credentials and associated attributes, and offering those to a verifier. Additionally, by using zero-knowledge proofs, data about the user that is not required by the verifier can be obscured – protecting end-user privacy. A combination of various signature schemes offers protection against replay and linking separate credential sharing sessions while letting the user have a final say in which attributes (e.g., that the user is over a certain age, and/or has a specific degree) are shared with the verifier.

A further advantage of this scheme is that these two steps can occur separately and in a more peer-to-peer fashion in the event of network disruption. This scheme is gaining adoption primarily in the Netherlands, and is spawning other research into, for example, verified identity social networks or those in which the anonymous user can post and sign the content to prevent it being changed after the fact as part of a disinformation campaign.

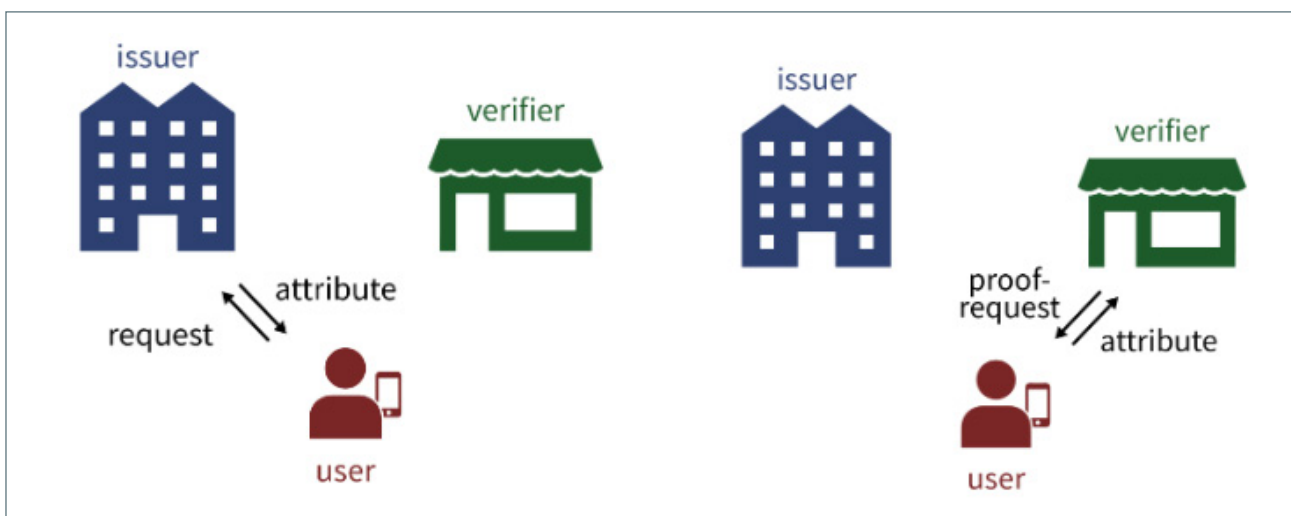


This scheme is gaining adoption primarily in the Netherlands, and is spawning other research into, for example, verified identity social networks.

TAKEAWAY:

- ✓ **Privacy-preserving online identity and attribute verification** can allow for many of the benefits of national ID schemes and more connected local governments without some of the risks. A seamless method to have a government-issued ID online without leaking unneeded attributes could pave the way for identified or anonymous networks, cutting down on spam and misinformation.

Figure 5: The high-level design goal of IRMA: allow for a user to have certain attributes attested by an issuer and then proven to a verifier – while allowing the user to protect their privacy by hiding unneeded attributes.



CryptPad: a zero knowledge collaboration platform

Author: Ludovic Dubost



[Code](#)



[Video](#)



[Website](#)

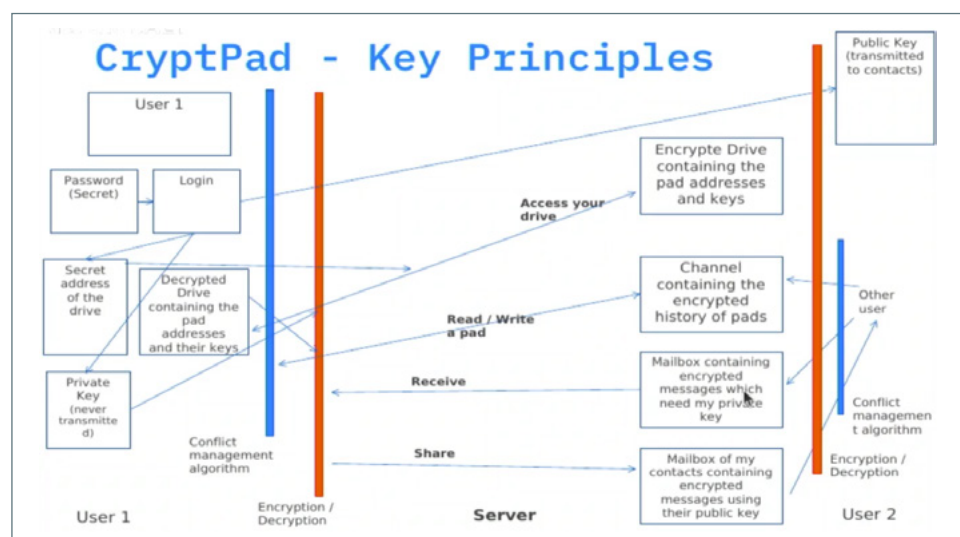
This talk is about the design, development and usage metrics of the open-source collaboration suite CryptPad. CryptPad is a suite of web-based tools that allow for collaborative editing and the creation of documents, spreadsheets, code snippets, etc., all the while being end-to-end encrypted. Using client-side cryptography in the browser, the server acts mostly as a storage service and message router for encrypted blobs. The figure below shows the process for logging in (generating a private key from the username and password) and sharing a document with another user to view or edit.

The end of the presentation covers adoption metrics and interesting trends in which countries are more aggressively migrating to this service. Finally, the presentation covered the business side of developing an open-source product (the CryptPad developers are also behind the XWiki project).

TAKEAWAY:

- ✓ **Ease of use is a crucial feature for privacy/security-enhanced tools**, especially those that are designed for user communication or collaboration. The adoption of these tools is governed by network effects – even if there is a privacy-minded user who is an advocate for a tool, if the others who must use it find it too clunky or difficult, it will not gain the momentum needed. The usage metrics presented about CryptPad show that there is enough usage to mature the UX and that it should continue to gain and retain users. Providing an alternative to ad-supported collaboration and communication suites can shift the amount of data exposed by both users and business entities to the larger service providers.

Figure 6: A flow for how to share a collaborative document from one user to another while retaining the end-to-end encryption – preventing the server from gaining access to the document content.



ARTICLE 1

Author: Yolan Romailler



Code



Video

drand: publicly verifiable randomness explained

ARTICLE 2

Author: Yolan Romailler



Code



Slides

A dead man's full-yet-responsible-disclosure system

In the first of these two related works, the researcher presents a distributed system that can provide public random values. This system ensures that, unless there is a malicious majority of network nodes, the random values cannot be biased or influenced. This is then built on in the second work to provide the novel capability of an encryption primitive, which acts like a time-lock safe, but without expensive and estimation-based proof-of-work techniques. drand is a system that allows the multiple parties which contribute to the network to input entropy from various sources (including true random number generators) and together generate a value every round (every 30 seconds) that can be verified and known as fair. Due to the thresholding scheme employed, a majority of the nodes (each run by a different organisation) would have to be tampered with to alter the agreed-upon random value.

Building on this in the second presentation, the author and their team are able to build a time-lock encryption scheme that allows for the protection of data until a specified future date and time. There are many proposed applications that use this primitive, including: managing embargoed documents, publicly posting a time-locked coordinated vulnerability disclosure to prevent any influence from stopping disclosure after the notification window, and sealed-bid auctions. While the drand feature required for time-lock encryption is not yet live, it should be in production later this year.

TAKEAWAYS:

- ✓ **If these protocols withstand scrutiny** as well as continue to build momentum and confidence that they will still be in existence in the years to come, the time-lock capability is a powerful primitive that will enable a multitude of interesting applications.
- ✓ **The idea of using a public-benefit distributed system** to replicate public-benefit single maintainer services (e.g., NIST's time beacons) is worth following. If some public entities joined with private internet good samaritans, it would decrease the risk of either unilateral subjugation for national interests or monetizing the service. DNS and some of the other technologies underpinning the internet show that there are broad benefits to federated services, and allowing users to migrate can limit a bad actor's impact (e.g., DNS servers directing clients to portals or other ads).

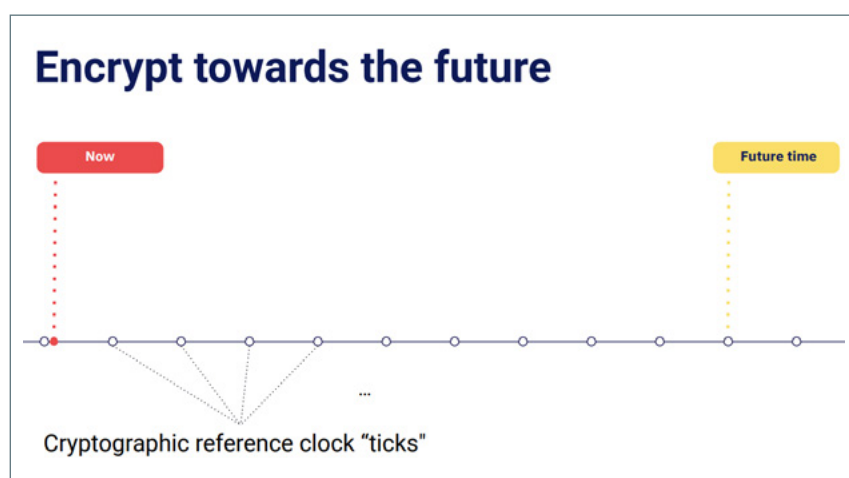


Figure 7: A simple diagram of the goal of encrypting data towards a clock tick in the future.

ARTICLE 1

Authors: Simon Rohlmann, Christian Mainka, Vladislav Mladenov, and Jörg Schwenk

[Slides](#)[Paper](#)

Oops... Code Execution and Content Spoofing: The First Comprehensive Analysis of OpenDocument Signatures

ARTICLE 2

Author: Alex Ivkin

[Code](#)[Video](#)

My data in your signed code

ARTICLE 3

Author: Golan Cohen

[Video](#)[Blog](#)

Can You Trust a File's Digital Signature? New Zloader Campaign exploits Microsoft's Signature Verification

These three research contributions highlighted the issues with the current state of digital signatures and how edge cases allow for untrusted data to be bundled with signed data. The first two explored how different programs and system components handled data that was countersigned (i.e., signed by multiple certificates). In the OpenDocument research, the diversity of readers for the open format resulted in a diversity of behaviours when confronted with the non-standard signed files. In some applications, one certificate was used to validate that the macros were signed and thus allowed to run, while the other certificate was used to ensure that there was a valid trust chain to the document. This allowed a self-signed attacker certificate to sign macros to run that would appear trusted by a legitimate certificate. Similar behaviour exists in Authenticode-signed code, where extra, unsigned data can be wrapped into a signed binary inside of an attacker signature. While this trick doesn't allow for direct execution of the added data, the last research entry showcases how that is possible.

In the third presentation, the author notes a strange finding in an active malware campaign: that the malware is apparently loaded only from signed code modules. As they dug further, they were able to see that the signed mshta.exe application was running scripts appended to another Microsoft-signed DLL. In 2012, it was discovered that the code signature checks would still validate if arbitrary data were appended to the signed code. This was fixed for a brief window, then it was reverted back to the [currently] vulnerable setting in 2014, unless certain registry keys are set. By updating the size of the signature and file checksum, malicious data can be appended to the signature section of the code file and the signature validation will still be successful.

TAKEAWAYS:

- ✓ **As application whitelisting and other controls on execution proliferate into modern operating systems**, attackers will be incentivised to explore these types of piggy-backing attacks and exploit the additional complexity in signature and certificate verification.
- ✓ **The end-user interfaces for understanding what is trusted**, why it is trusted and by whom, are lacking in maturity. It is a near-impossible task for a user to determine whether a file or program is legitimate and trusted by their organisation, or simply one in the forest of intermediate certificate authorities.

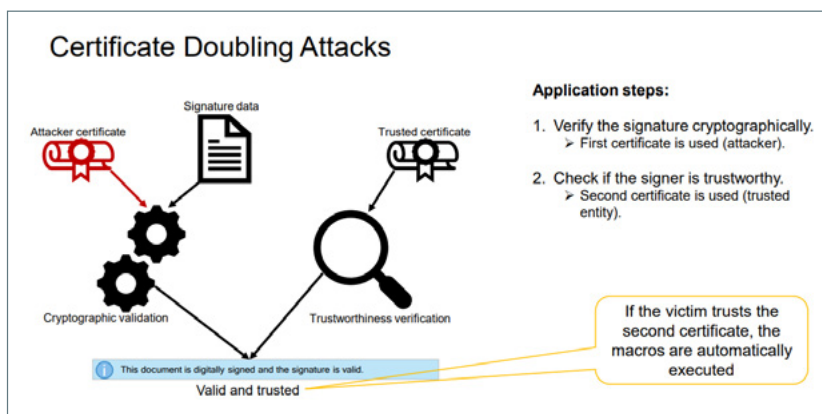


Figure 8: A high-level diagram of how some OpenDocument readers can be confused by documents signed with multiple certificates.



Software analysis at scale

- ✓ *TLS-Anvil: Adapting Combinatorial Testing for TLS Libraries*
- ✓ *Arbiter: Bridging the Static and Dynamic Divide in Vulnerability Discovery on Binary Programs*
- ✓ *In Need of 'Pair' Review: Vulnerable Code Contributions by GitHub Copilot*
- ✓ *Catch Me If You Can: Deterministic Discovery of Race Conditions with Fuzzing*
- ✓ *Someone's Been Messing With My Subnormals!*

Photo by Stephan Louis on Unsplash.

TLS-Anvil: Adapting Combinatorial Testing for TLS Libraries

Authors: Marcel Maehren, Philipp Nieting, Sven Hebrok, Robert Merget, Juraj Somorovsky, and Jörg Schwenk



[Code](#)



[Slides](#)



[Website](#)

While TLS is considered a security baseline, flawed implementations undermine the promised security properties. Such implementation flaws result from the TLS specifications' complexity, as well as the exponential number of possible parameter combinations. This paper develops both tooling and a methodology for testing TLS libraries. Both of these help answer the questions, "Is the observed behaviour correct for a given input?" and, "How well do current TLS libraries perform in regards to security, interoperability and conformance, considering the complexity of requirements from the TLS specification and scientific literature?"

The open source TLS-Anvil tool has been published for the community to use and learn from.

TAKEAWAY:

- ✓ **Testing cryptographic software is complicated and complex.** This paper presents tooling and methodology to help ensure the security promises of a given implementation are upheld.



... flawed implementations undermine the promised security properties. Such implementation flaws result from the TLS specifications' complexity, as well as the exponential number of possible parameter combinations.

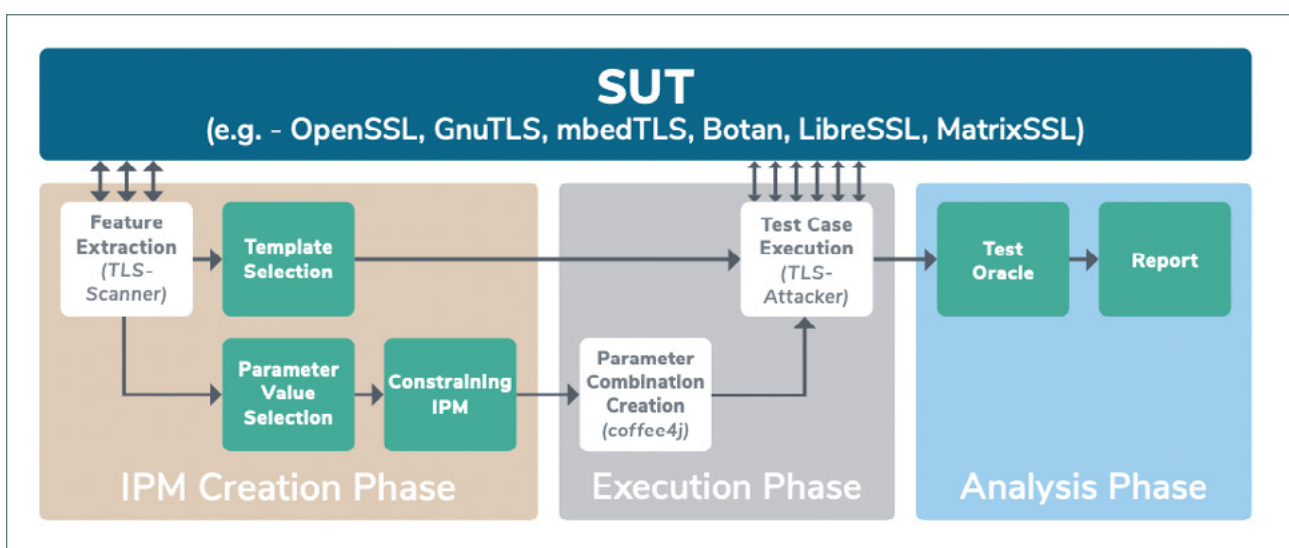


Figure 9: A diagram showing how TLS-Anvil performs its testing on the target TLS library (Software Under Test (SUT)). Elements marked in green depict the contributions of this paper.

Arbiter: Bridging the Static and Dynamic Divide in Vulnerability Discovery on Binary Programs

Authors: Jayakrishna Vadayath, Moritz Eckert, Kyle Zeng, Nicolaas Weideman, Gokulkrishna Praveen Menon, Yanick Fratantonio, Davide Balzarotti, Adam Doupé, Tiffany Bao, Ruoyu Wang, Christophe Hauser, and Yan Shoshitaishvili



[Code](#)



[Paper](#)

This research looks at how best to combine the benefits of static and dynamic analyses into a single workflow while minimising the downsides of each. Popularised by DARPA's Cyber Grand Challenge (CGC), the combination of static and dynamic analyses into a single toolchain has resulted in significant improvements. Arbiter targets Linux binaries (whereas CGC tools targeted a simplified fake OS called DECREE) and does not need either source, or for the binaries to be instrumented (or even runnable). A vulnerability description is defined for the specific flows within a program that would be vulnerable—the description codifies the data flows and other constraining invariants to reduce the false positives from static over-approximation.

Arbiter focuses on vulnerabilities that:

1. are sensitive to data-flow characteristics,
2. have known sources and/or sinks for those flows, and finally,
3. have aliasing determined by control-flow to prevent needing the full application context.

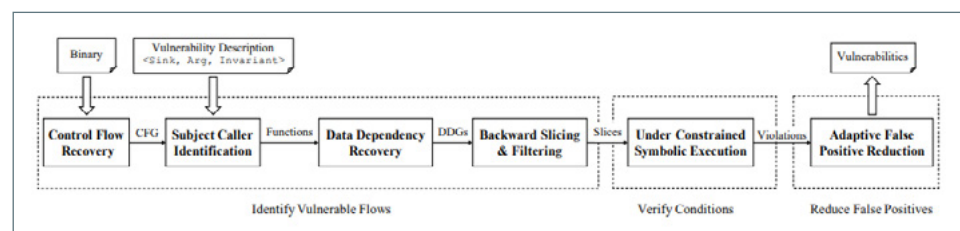
First the control-flow graph is extracted from the binary, then a subgraph is collected between the source(s) and sink(s) that are related by data-flow. The constraints in the description are used to further filter candidates that are then tested dynamically using a technique called under-constrained symbolic execution that will symbolically execute a slice of a program even without all the data structures initialised (the technique assumes that pointers are valid). Finally,

possibly-vulnerable code is potentially re-executed with more context to improve the performance of the toolchain, trading off performance for accuracy. An analysis of ~76k binaries from the Ubuntu 18.04 repositories found 1095 possible vulnerabilities across four CWE descriptions. Manual verification determined that more than half were true positives (since reported), with the remainder either definite false positives or undetermined due to the complexity of the application.

TAKEAWAYS:

- ✓ **As open-source cyber-reasoning systems mature**, the simplicity of codifying a vulnerability into a more generic CWE-style description will tighten the timelines on similar bugs being found at scale. Tracking CVEs is no longer sufficient as a method to close the loop in software security – machine readable descriptions like Arbiter's, CodeQL's, etc. are needed to keep pace with the volume of software development across the world.
- ✓ **The ability to scan for vulnerabilities** across an entire operating system's software ecosystem is a game changer. Similar to how tools like MASSCAN or ZMap can be used to measure internet-wide statistics about vulnerability exposure and patching, tools like Arbiter can track the improvement of software across the whole ecosystem over time.

Figure 10: A diagram of the steps Arbiter takes on binaries to test and vulnerability descriptions to identify vulnerabilities in real-world software.



In Need of 'Pair' Review: Vulnerable Code Contributions by GitHub Copilot

Authors: Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, and Baleegh Ahmad



[Paper](#)



[Slides](#)

This research explored the security of the code generated by the AI assistant GitHub CoPilot. CoPilot and the other AI programming assistant tools advertise higher productivity by allowing an AI model trained on extensive code corpi to generate code within a developer's IDE. Going beyond auto-complete or other IDE integrations, these assistants can work off a natural language prompt (e.g., `// Function to process username and password for login`) and generate functional code in larger quantities than previous helpers. The authors explored some of the generated code snippets to a simple prompt, and noticed a trivial SQL injection vulnerability in the AI-generated code. In order to scale up and measure the prevalence of such introduced vulnerabilities, the research team needed a way to check a large set of code samples for the presence or absence of a vulnerability.

The figure below shows the framework to scale beyond manual analysis – using CodeQL to statically check for specific CWEs that would be relevant to code generated by a number of prompts. The prompts also could be mutated to see if the wording or other small syntactic changes would influence the output quality, as well as understanding how meta-data such as author name in comments would impact security. The framework was used to explore the generation of almost 2000 programs across three languages: Python, C, and an HDL. Overall almost 40% of the top responses (which a typical developer would use) were vulnerable to the expected CWE. The authors finish with a recommendation to ensure that CoPilot and other assistants are used only as a support tool, and not a replacement for development and security knowledge.

TAKEAWAYS:

- ✓ **As tools like CoPilot improve,** AI-generated code will comprise an increasing portion of code-bases. If those code-bases are open-sourced, they will be used to train the models for the next iteration of CoPilots. Until there are better safeguards in place, the most prolific code generators will have an outsized influence on software quality across the industry.
- ✓ **There are other drawbacks to these types of systems:** while increased speed of generating new code is a plus, in addition to the security concerns, there is a loss of understanding of the codebase by its own developers, which will make future modifications more difficult.

Experimental Framework

Manual analysis does not scale!

Pair Copilot with static analysis - **GitHub CodeQL!**

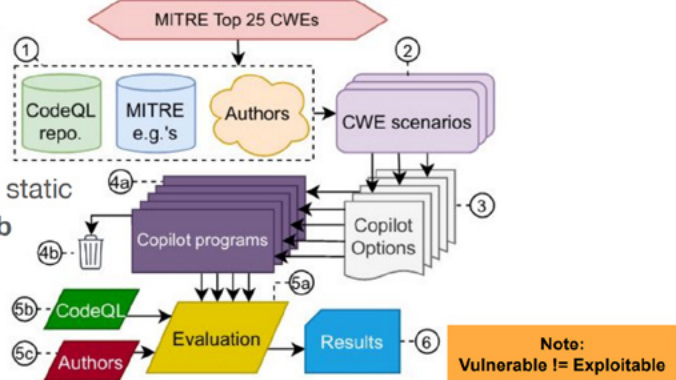


Figure 11: A high-level diagram of the experimental setup.

Catch Me If You Can: Deterministic Discovery of Race Conditions with Fuzzing

Author: Ned Williamson



[Code](#)



[Slides](#)

This research looked to provide the foundation for more reliable and effective fuzzing of multi-threaded race conditions. While traditional fuzzers and testing can sometimes unearth the right sequence of inputs and get lucky with the scheduling to find these types of bugs, it is difficult to reproduce. In this work, the researcher aimed to convert the target to one using cooperative multithreading where the transitions between threads are deterministic as opposed to stochastic. In this way, bugs are reproducible, and part of the mutable state the fuzzer explores is where these transitions occur.

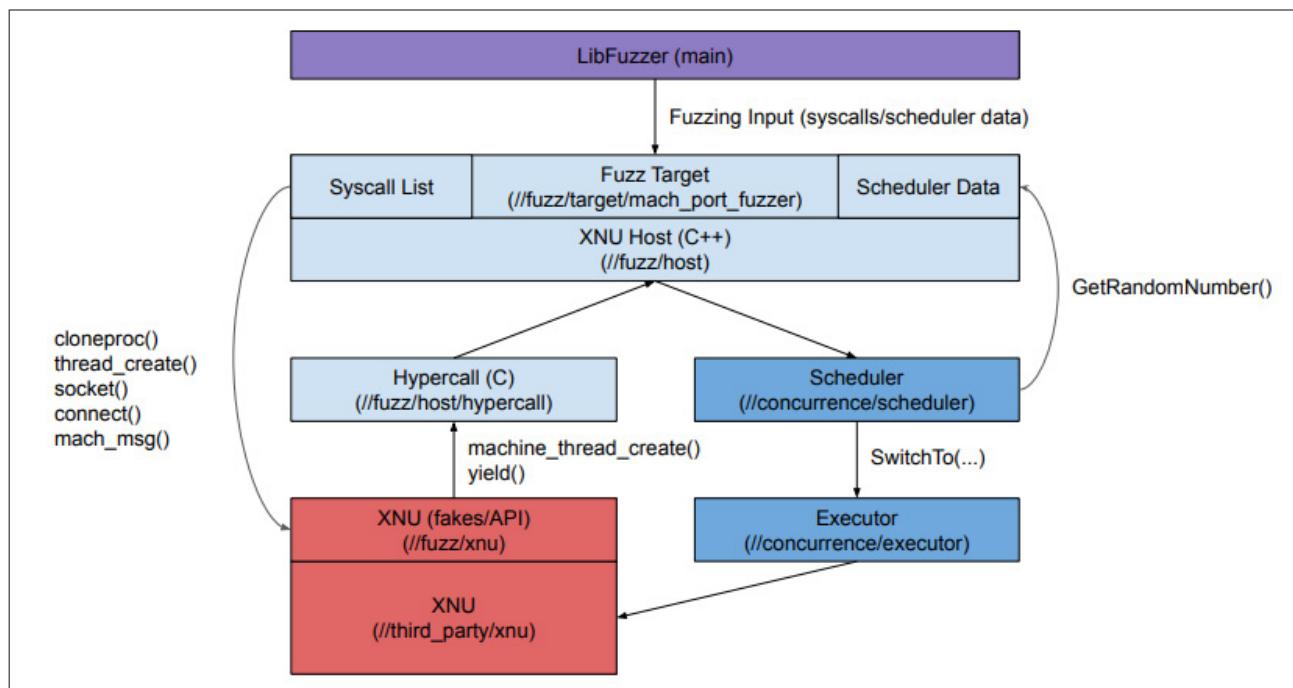
While there is an uncountable state space for all possible interactions between multiple threads, this work allows for existing research on guiding fuzzers to include these interactions as opposed to randomly stumbling on them. In addition to the improvements to the fuzzer, a number of real-world bugs are explored – this approach is paying real dividends for bug hunters.

TAKEAWAY:



Despite the substantial research effort in the fuzzing community, it is a relative rarity for a new class of vulnerabilities to be detectable. As this work is integrated into existing frameworks, it will allow for more effective use of the CPU cycles by covering more possible vulnerability types. As more classes of vulnerabilities are covered by programs like OSS-Fuzz, stronger security assurances can be made about the components underpinning most software systems.

Figure 12: A diagram showing how the improved SockFuzzer fits into the MacOS environment.



Someone's Been Messing With My Subnormals!

Author: Brendan
Dolan-Gavitt



Blog

Figure 13: An abbreviated table of the most popular Python packages that indirectly import a library that is built with `--ffast-math`.

Digging into a warning about floating point subnormals (floating point values that are very close to 0) being replaced with zero (a behaviour that can break certain numerical algorithms), the researcher ended up exploring the Python dependency ecosystem. Due to the design of floating-point acceleration hardware, the floating point behaviour is set in a CPU register for an entire thread or process, so a single dependency that enables it via a compilation flag will change the behaviour of the entire process. Looking at his Python process, there were over 150 libraries loaded, of which one (or more) was changing the mathematical results for all of them.

The researcher was curious about how prevalent this behaviour was across the Python ecosystem, especially in popular packages. PyPi publishes a table of all packages available along with their usage metrics, and with a simple script to look for the machine code that enables the floating-point optimization, this pipeline could identify if a library was causing this issue. Next the researcher ran this pipeline on the entirety of the PyPi repository, both consuming a considerable amount of bandwidth and disk space, and running untrusted `setup.py` scripts from every package (one of which tried to run the `sudo` command). With a list of all packages that enable this

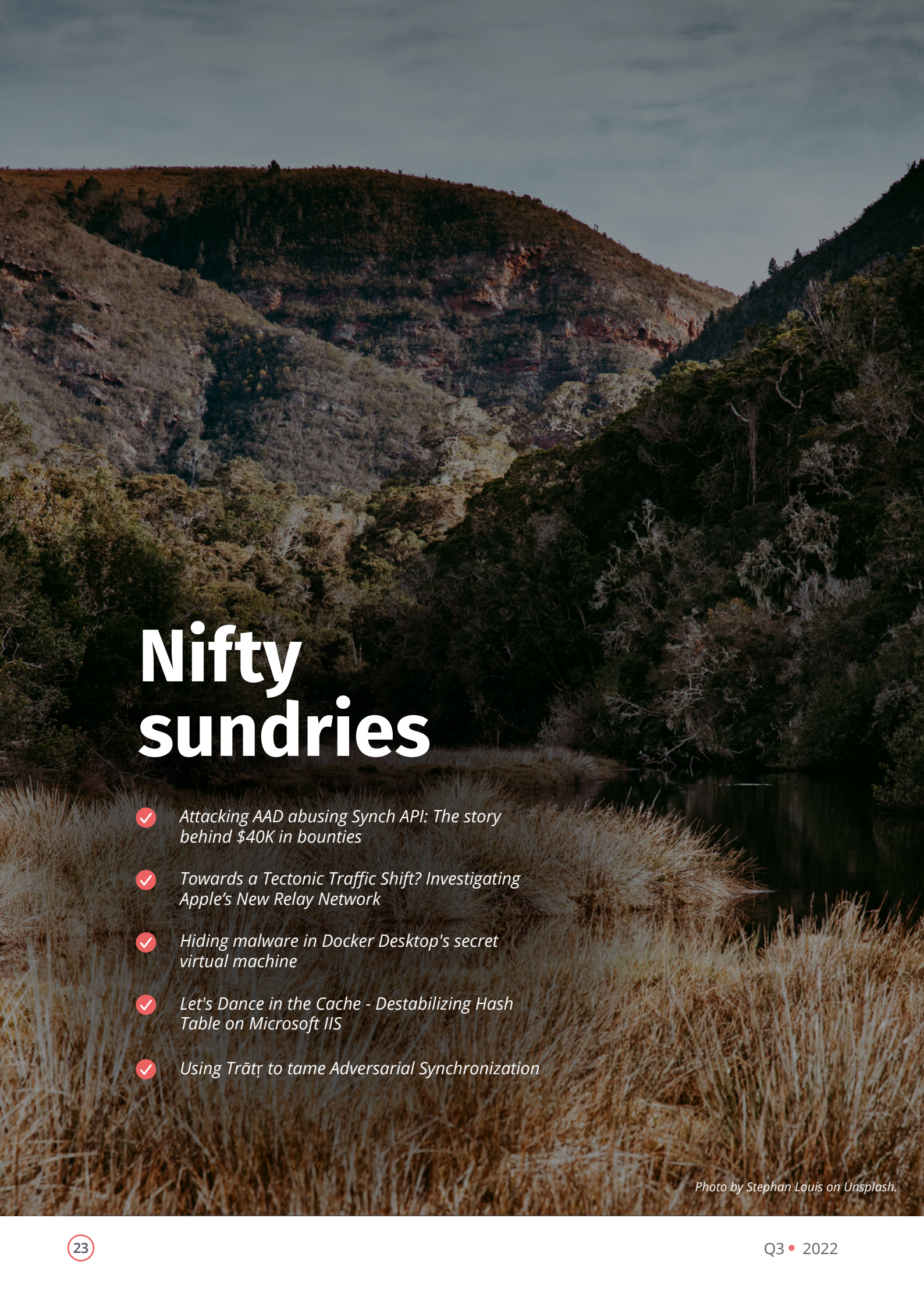
behaviour, the last step was to look for libraries that imported any of these packages.

Google's deps.dev provides a sample of packages that depend on a target, so a wider list of imports that would trigger this behaviour was gathered, and those in the top 3000 of popular packages are shown in the figure to the left.

project	source	num_downloads	rank
geventhttpclient	gevent	1116639	1296
locust	gevent	1045789	1345
flask-socketio	gevent	914846	1424
dagster	gevent	497982	1974
grequests	gevent	365292	2328
dedupe	btrees	358737	2352
websocket	gevent	346515	2397
gevent websocket	gevent	322249	2466
dagster-graphql	gevent	310382	2525
locust-plugins	gevent	273452	2669
interpret-community	gevent	268129	2692
zope-index	btrees	240577	2836
dedupe-variable-datetime	btrees	223080	2932
parallel-ssh	gevent	220882	2947

TAKEAWAYS:

- ✓ **These “contagious” libraries that change the behaviour** of other third party or even first party code are another threat of the deep dependency trees. A more rigorous exploration of how an imported library can alter behaviour of co-loaded code would likely uncover other subtle ways in which libraries that are never directly executed can subvert software.
- ✓ **The amount of effort that the author went through** to answer these types of questions highlights the challenges facing concerned end-users who are trying to understand their software supply chain. Most of the hype regarding SBOMs misses the build and compilation information that would capture this type of issue – hopefully a future iteration of software supply chain management tools will take this into account.



Nifty sundries

- ✓ *Attacking AAD abusing Synch API: The story behind \$40K in bounties*
- ✓ *Towards a Tectonic Traffic Shift? Investigating Apple's New Relay Network*
- ✓ *Hiding malware in Docker Desktop's secret virtual machine*
- ✓ *Let's Dance in the Cache - Destabilizing Hash Table on Microsoft IIS*
- ✓ *Using Trätṛ to tame Adversarial Synchronization*

Photo by Stephan Louis on Unsplash.

Attacking AAD by abusing the Sync API: The story behind \$40K in bounties

Author: Nestori Syynimaa



[Code](#)



[Slides](#)



[Video](#)

This research explored the process by which local, on-premise AD credentials are synchronised with an Azure AD directory. Azure AD Connect is a service that runs on the on-premise network with high privileges to push changes in the local environment to Azure. The credentials for this service are available to local system administrators, and can then be used from anywhere to change the passwords, display names, etc. for both local and cloud-only accounts. There was an implementation flaw that allowed for the modification and deletion of Azure AD global administrators – essentially locking an organisation out of their cloud resources.

After the flaw was addressed through the creation of a new directory flag to prevent local accounts from modifying cloud-only accounts, the researcher discovered that the local synchronisation accounts could change the directory flags to disable the fix. This was fixed again earlier this year.

TAKEAWAY:

- ✓ **Having multiple authoritative sources of credentials** exposes a variety of gaps for attackers. The handling of the report, and the subsequent discovery of a fix bypass does not inspire confidence in the overall security of a hybrid authentication environment. While the benefits of single sign-on from a usability perspective are clear, the complexity needed to seamlessly support the feature is likely to cause future issues. Keeping a single authoritative source may be safer for the time being.

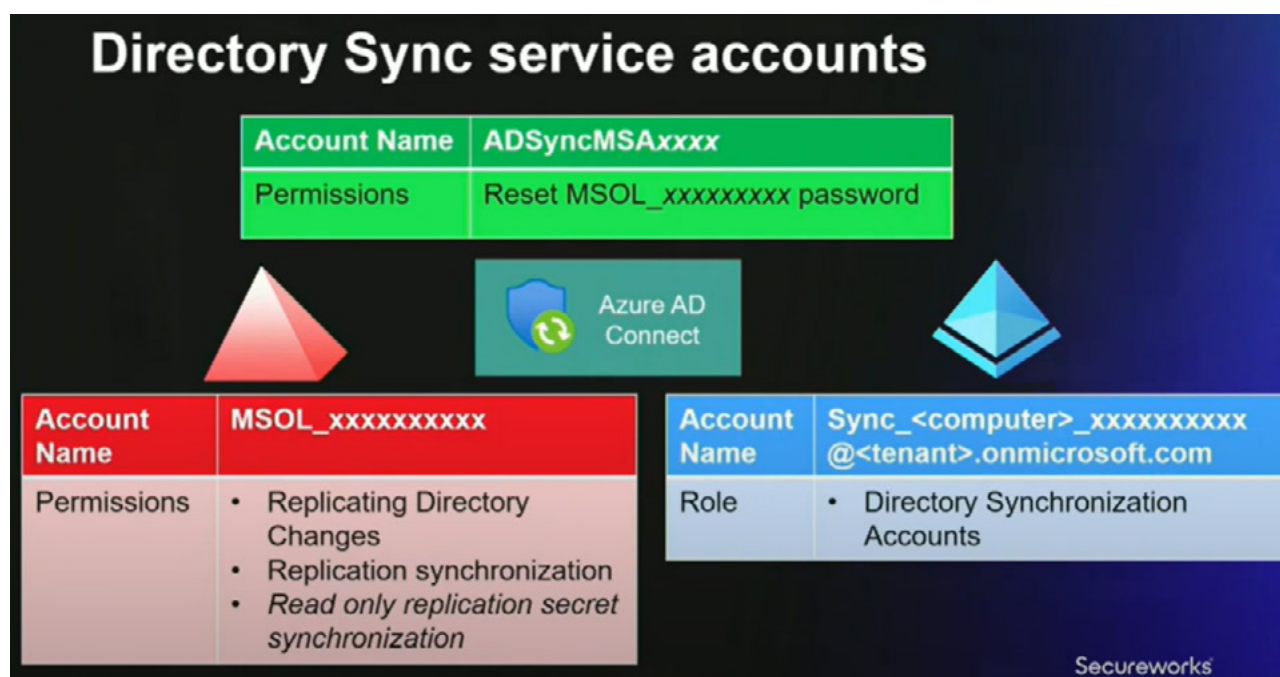


Figure 14: The three accounts and their associated permissions that underpin the synchronisation between Azure AD and an on-premise AD directories.

Towards a Tectonic Traffic Shift?

Investigating Apple's New Relay Network

Authors: Patrick Sattler,
Juliane Aulbach,
Johannes Zirngibl, and
Georg Carle



This paper takes a close look at the Apple iCloud Private Relay service, currently in beta. Their specific aim was researching the impact this service might have on network measurement-based research. The team was able to perform passive monitoring as well as ingress and egress observations to develop maps and models for the relay network. Additionally, they were able to identify changes in architecture as it was observed over time as well as the third party vendors supporting the service.

TAKEAWAYS:

- ✓ **iCloud Private Relay relies on raw public keys** instead of the usual certificate authentication within its TLS handshake. Deployed key pinning prevents the interception using TLS proxies, and hence an in-depth analysis of the protocol is infeasible. This may be an area for future research, both in terms of observing traffic and key material recovery.
- ✓ **This service is planned to be enabled by default**, once it is out of testing. This may have a large impact on traffic routing, performance bottlenecks, fraud detection, and other network characteristics.
- ✓ **The protection seems to focus on Safari and cleartext traffic**, which may leave opportunity for leaking original locations, given that not all traffic is routed through the private network. It was not clear if other applications can leverage this network.

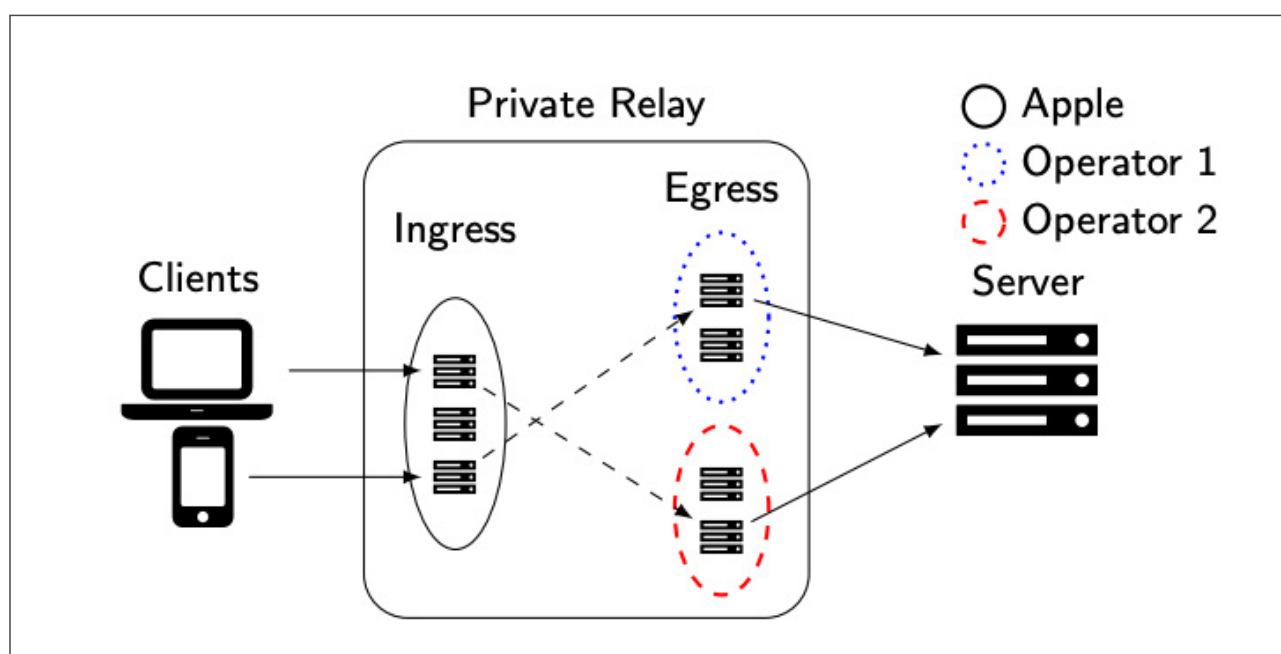


Figure 15: iCloud Private Relay Architecture.

Hiding malware in Docker Desktop's secret virtual machine

Author: Alex Hope



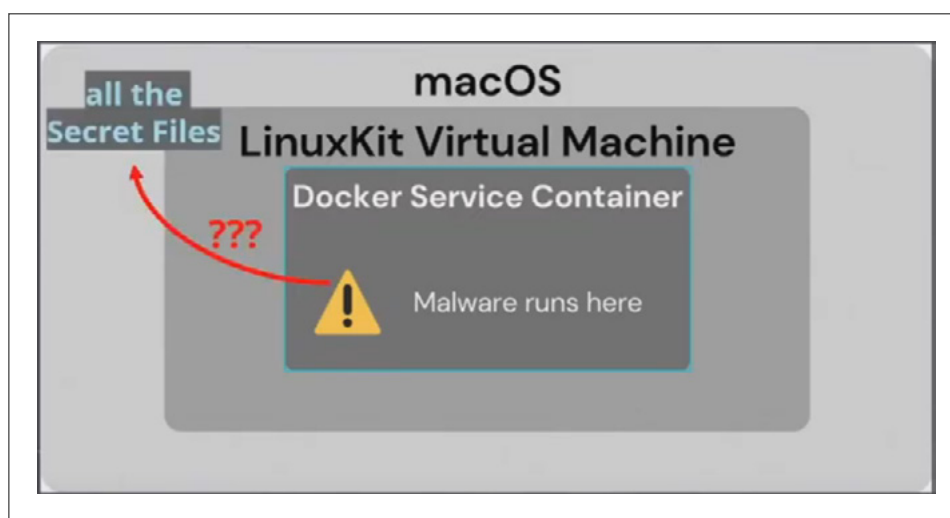
This research looked at how to persist on a target MacOS system for an extended period of time in a challenge called, "Don't get caught for a year." Looking at the software stack installed at their organisation, Docker Desktop stuck out. On non-Linux systems, Docker transparently installs a host Linux virtual machine to fork containers from. In this VM is a service container that provides a number of convenience features, and that container has a configurable runtime environment that is run in the service container. Arbitrary code then can be run within the container which, by default, has full access to the majority of the host OS file system.

While the code within the container has access to the host OS file system, it is difficult to introspect into the state of the container (e.g., running processes or syscalls). This means that malware running within the container does not need to add complexity in evasion or other anti-detection techniques.

TAKEAWAY:

- ✓ **The task of AV/EDR is already challenging** before adding in the semantic gaps created through multiple OSes and containers. While virtual machine introspection techniques could provide insights into the operations of the containers, ever more convoluted tools to "simplify" packaging and deployment of software will continue to obscure what is happening on your computer.

Figure 16: A diagram of where malware in the Docker Desktop container runs in a VM on MacOS. The container has access to the host MacOS file system.



ARTICLE 1 Author: Orange Tsai[Slides](#)[Blog](#)

Let's Dance in the Cache: Destabilizing Hash Table on Microsoft IIS

ARTICLE 2

Authors: Yuvraj Patel, Chenhao Ye, Akshat Sinha, Abigail Matthews, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Michael M. Swift

[Slides](#)[Paper](#)

Using Trätr to tame Adversarial Synchronization

A modern look at algorithmic attacks is presented in each of these works, the former looks at both a DoS and a remote authentication bypass in Windows IIS servers, and the latter looks at a DoS on shared data structures in the Linux kernel exposed to unprivileged containers. The first work shows three attacks against Microsoft's IIS caching capabilities: a DoS in the hashing algorithm which, with 30 requests-per-second, can turn a server unresponsive (1), a differential in how two components in the IIS stack handle identifiers to cache, allowing for an attacker to poison the cache for others (2), and a method to bruteforce the Basic Auth password validation scheme (3). Each had a slightly different implementation flaw that allowed the attacks to succeed, but the ecosystem of caching and other hash tables across the Microsoft IIS stack shows ample space for abuse.

The second looked at how unprivileged, multi-tenant Linux container applications could impact one another via synchronisation of shared resources. Specifically this work explored the critical sections of accessing hash tables with linked lists shared in the kernel. By creating a FUSE user-space filesystem, the researchers could create collisions in the global inode hash table resulting in inefficient allocations and very long search times while holding a global lock. This has a significant impact on the performance of the victim container[s] (see figure below). A true multi-tenant container environment is rare (usually a VM separates tenants), but this work highlights the risks of relying on the good behaviour of all processes when it comes to synchronisation.

TAKEAWAYS:

Algorithmic attacks are often overlooked as low-impact or difficult to put into practice, but these publications show how a DoS attack can be made impactful (e.g., in a multitenant container environment) or even go beyond DoS to access violations. It's likely that the vast majority of data structures courses hit upon hash collisions as a non-malicious concern, and therefore software is not built around the asymptotic worst-case.

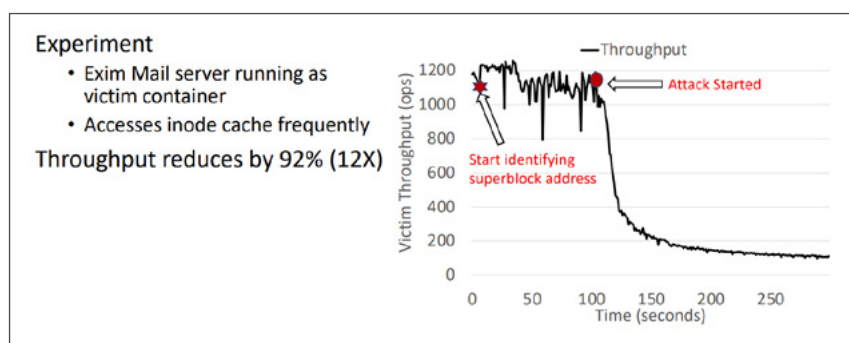


Figure 17: A graph of throughput for a mail server running on another container after the introduction of hash-collisions in the inode table.

Conclusion

This quarter covered a large amount of research publication venues – the sheer quantity of published work to review speaks for itself. In the course of the past year of ThinkstScapes, readers have seen new research areas sprout from one or two papers into full-blown themes with growing momentum.

THREE THEMES WERE HIGHLIGHTED IN THIS QUARTER'S EDITION:

1. AI/ML boosted side-channels and data analytics.
2. Clever cryptography and some subtle bugs in signatures.
3. Scaling the breadth and depth of automated software analysis.

Next quarter will likely see a reduction in quantity with the holidays, but the amount of high-quality blog posts is unlikely to slacken. Check back next quarter for insights into work that is well-publicised, as well as highlights that may have been missed.

Photo by Wolfgang Hasselmann on Unsplash.

